Prezime i ime učenika : *Mihajlo Stoiljković*

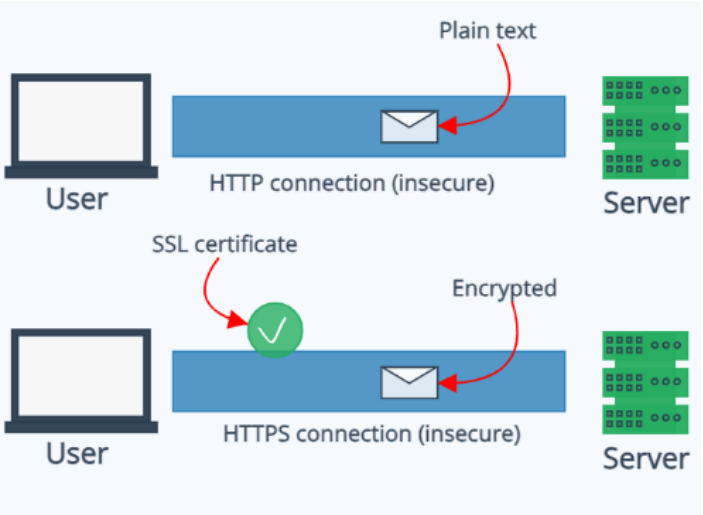
Odeljenje i grupa : *IV-4 grupa 3.*

R. broj i naziv vežbe : *Покретање једноставног VEB сервера и управљање периферијама на Raspberry Pi*

IZVEŠTAJ:

HTTP (Hyper Text Transfer Protocol) je web protokol koji služi za komunikaciju odnosno prenos podataka između klijenta (npr. neke aplikacije ili web pregledača) i servera na webu. Pomoću ovog protokola klijent šalje zahteve serveru, a server kao odgovor šalje tražene podatke kao što su neke web stranice ili podaci uz pomoć nekog API-a (**WeatherAPI**...) odnosno nekog drugog sadržaja. Ono što je karakteristično za http prokol jeste što je u pitanju jednosmerna komunikacija (istovremeno može samo klijent ili server da salje podatke), i svaki klik obuhvata nov http zahtev što ga čini sporijim protokolom pa se zato često koristi za učitavanje i prikaz web stranice.

Pored http protokola imamo https (Hyper Text Transfer Protocol Secure) je sigurnosna verzija http protokola koji se koristi često za web stranice kod kojih se vrši upis podataka unutar neke baze podataka ili slično. Time se dobija sigurnosno slanje podataka do odredjene baze ili servera čiji i jeste princip rada jer šalje podatke u ekriptovanom obliku.

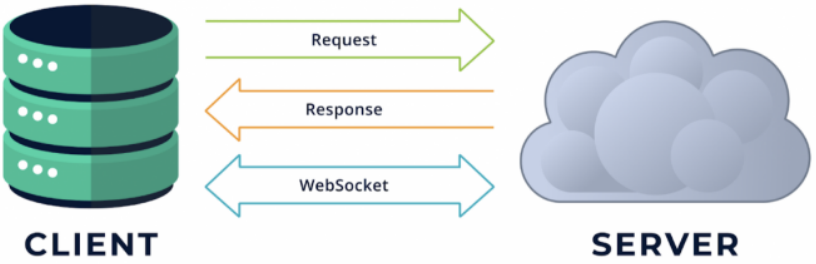


Web socket je web tehnologija za stalnu dvosmernu komunikaciju izmedju klijenata (Aplikacije, web browsera) i servera preko mreže. On omogućava da klijent i server stalno komuniciraju u realnom vremenu jedan s drugim, bez stalnog slanja novih zahteva. Jednom se otvori veza i ona traje dok se komunikacija ne završi što ga čini znatno bržim u odnosu na HTTP/S protokol. Pogodan je kada je potrebno RealTime dobijanje podataka od servera odnosno kada je prioritet brzina, ali se treba znati da je sama integracija teža nego kod HTTP protokola. Koristi se često kod:

- IoT uređaji (Arduino, Rpi...)

- Online igre
- Chat aplikacije
- Pametne kuće
- Praćenje senzora u realnom vremenu..

Što se tiče same vežbe/projekta, potrebno je da se odradi na dva načina i to koristeći Http protokol kao i WebSocket. Cilj jeste da uz pomoć jedne web stranice omogućimo korisniku upravljanje radom jedne Led diode na raspberry PI mikroračunaru.



Kreiranje i upravljanje Web Serverom uz pomoć HTTP protokola

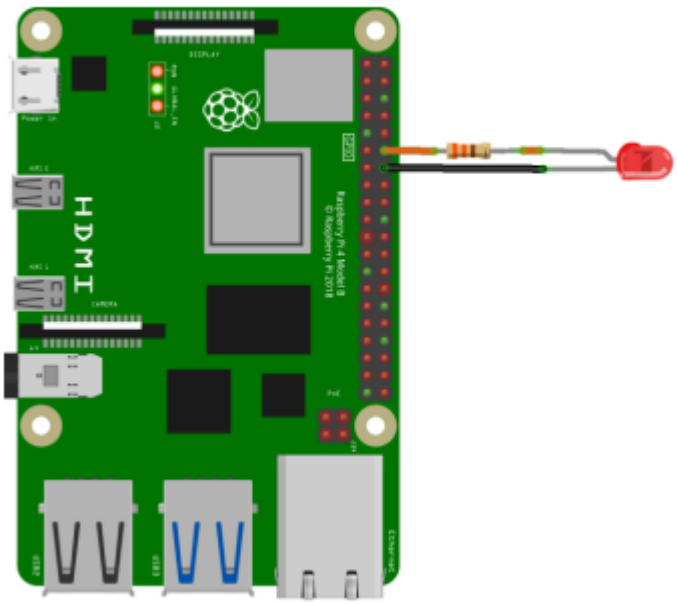
Struktura projekta za HTTP:

projekat/

├── server.py

├── index.html

Povezivanje:



Za sam http nije potrebno instalirati ni jednu bibloteku na linux-u jer sve već postoji. Python skripta za pokretanje web servera:

```python
# server.py
import RPi.GPIO as GPIO
from http.server import BaseHTTPRequestHandler, HTTPServer
LED_PIN = 12 # BCM pin
# GPIO podesavanje
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.output(LED_PIN, GPIO.LOW)
class WebServer(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/on":
            GPIO.output(LED_PIN, GPIO.HIGH)
            self.send_response(200)
            self.end_headers()
            self.wfile.write(b"LED ON")
        elif self.path == "/off":
            GPIO.output(LED_PIN, GPIO.LOW)
            self.send_response(200)
            self.end_headers()
            self.wfile.write(b"LED OFF")
        else:
            self.send_response(200)
            self.send_header('Content-type', 'text/html')
            self.end_headers()
            # Otvaranje index.html fajla
            with open("index.html", "rb") as file:
                self.wfile.write(file.read())
try:
    # Kreiranje instance server
    server = HTTPServer(("0.0.0.0", 8080), WebServer)
    print("Server pokrenut na portu 8080")
    server.serve_forever()
except KeyboardInterrupt:
    print("Server zaustavljen")
finally:
    GPIO.cleanup()
    server.server_close()
```

Html web stranica za korisničko kontrolisanje diode uz pomoć dva dugmeta:

```html
<!-- index.html -->
<!DOCTYPE html>
<html lang="sr">
<head>
```

```
<meta charset="UTF-8">
      <title>LED kontrola - Raspberry Pi</title>
</head>
<body>
      <h2>Upravljanje LED diodom</h2>
      <button onclick="location.href='/on'">UKLJUČI LED</button>
      <button onclick="location.href='/off'">ISKLJUČI LED</button>
      <p>Otvoreno sa Raspberry Pi veb servera</p>
</body>
</html>
```

Nakon toga sledi pokretanje web servera sa rasp pi terminala:

```
mihajlo@DESKTOP-BAFHUP9:~$ python3 server.py
```

Pristup iz web browsera:

http://IP_ADRESA_RASPBERRY_PI:8080

(npr. http://192.168.1.50:8080)

Funkcionalnost projekta:

Na početku, objasnićemo šta radi python skripta za server. Importujemo biblioteku za Rpi kao i http.server modul odnosno **BaseHTTPRequestHandler, HTTPServer klase** iz modula zbog http komunikacije sa serverom. Nakon podešavanja gpio pina i postavljanje na logički nivo 0 (isključena dioda), kreiramo klasu WebServer koja nasleđuje klasu **BaseHTTPRequestHandler** koja poseduje metode za rad sa http prokolom (response, headers...). Kreiramo metodu do_GET koja služi za dobijanje datog responsa odnosno odgovora. Neki od status kodova odnosno responsa su:

✅ 2xx – Uspešni odgovori

- **200 OK** – zahtev je uspešno izvršen
- **201 Created** – uspešno je kreiran novi resurs

🔁 3xx – Preusmerenje

- **301 Moved Permanently** – trajno preusmerenje
- **302 Found** – privremeno preusmerenje

❌ 4xx – Greške klijenta

- **400 Bad Request** – loše poslat zahtev
- **401 Unauthorized** – nije izvršena autorizacija
- **403 Forbidden** – pristup zabranjen
- **404 Not Found** – traženi resurs ne postoji

⚠ 5xx – Greške servera

- **500 Internal Server Error** – greška na serveru

- **502 Bad Gateway** – loš odgovor drugog servera
- **503 Service Unavailable** – servis trenutno nedostupan



U ovom delu možemo videti poruku u thonny okruzenju u shellu da je server pokrenut, kao i dešavanja koja su se dogodila odnosno odgovori od servera (GET) sa datim kodom. Unutar metode do_GET() imamo uslove za dve moguće putanje koje šalje klijent odnosno neka web stranica a to su:

- **/on** -> dioda uključena
- **/off** -> dioda isključena

Ukoliko je putanja **/on** onda postavljamo diodu na logički nivo 1 i šaljemo response 200 odnosno OK response i ispisujemo LED ON. Ukoliko je putanja dobijena od klijenta **/off** onda se stavlja dioda na logički nivo 0 I ispisuje se LED OFF. Ukoliko nista nije poslato, šalje se takodje response 200 i vrši se otvaranje index.html fajla i unutar njega se ispisuje na web stranici podaci koji su pisani za stanje diode. Nakon toga, vršimo obradu izuzetaka gde u try bloku kreiramo instancu **HTTPServer** klase koja sadrži konstruktor od dva argumenta gde prvi argument predstavlja adresu I port servera, a drugi argument je klasa koja obradjuje HTTP zahteve kod nas je to klasa **WebServer**. Što se tiče adrese servera možemo staviti lokalnu petlju ili adresu koja je dostupa I drugim uređajima u mreži:

- `127.0.0.1` → samo lokalno
- `0.0.0.0` → dostupno i drugim uređajima u mreži

Nakon toga printujemo da je server pokrenut na portu 8080 kao i to da serviramo server zauvek odnosno pokretanje HTTP servera koji stalno radi i osluškuje dolazne zahteve klijenta. Svaki klik na dugme pravi nov http zahtev.

U catch bloku, ako se desi KeyboardInterrupt izuzetak, ispisujemo da je server zaustavljen, a finally blok se bez obzira da li se uhvatio izuzetak ili ne izvršava i vrši postavljanje GPIO pinova na stanje visoke impedanse i zatvaranje HTTP servera.

Nakon objašnjena python skripte, sledi objašnjenje html strukture odnosno koda:

Kao I svaki html kod, sastoji se iz osnovne strukture. Sastoji se iz jednog headinga, jednog paragrafa I dva dugmeta koji kao event **onclick** postavljaju putanju /on za uključenje LED diode i /off za isključenje. Nakon toga, u python skripti se čitaju podaci sa html-a i na osnovu dobijene putanje se šalje dati response i izvršava odredjena radnja.



Kreiranje i upravljanje Web Serverom uz pomoć WebSocket-a

Struktura projekta za WebSocket:

projekat/

├── ws_server.py

├── index.html

Što se tiče povezivanja, isto je kao i kod prvog dela projekta. Kako bi koristili WebSocket u našem projektu, potrebno je izvršiti instalaciju biblioteke websockets.

```
mihajlo@DESKTOP-BAFHUP9:~$ pip3 install websockets
```

Nakon instalacije biblioteke, možemo napisati python skriptu:

```python
# ws_server.py
import asyncio
import websockets
import RPi.GPIO as GPIO
LED_PIN = 12
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.output(LED_PIN, GPIO.LOW) # Postavljanje led-a na 0 logicki nivo
async def led_control(websocket, path=None): # Asinhrona f-ja
    async for message in websocket:
        if message == "ON": # Ukoliko je poruka ON ukljuci led
            GPIO.output(LED_PIN, GPIO.HIGH)
            await websocket.send("LED UKLJUČENA")
        elif message == "OFF": # Ukoliko je poruka OFF iskljuci led
            GPIO.output(LED_PIN, GPIO.LOW)
            await websocket.send("LED ISKLJUČENA")
async def main():
    async with websockets.serve(led_control, "0.0.0.0", 8765):
        print("WebSocket server pokrenut na portu 8765")
        await asyncio.Future() # radi stalno
try:
```

```
    asyncio.run(main())
except KeyboardInterrupt:
    print("Server zaustavljen") # Kada se server zaustavi
finally:
    GPIO.cleanup() # Vracanje pinova na stanje visoke impedanse
```

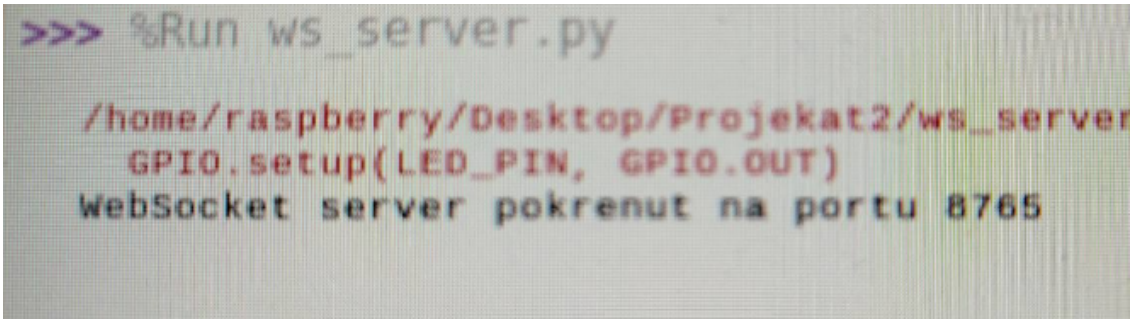
Html/JavaScript stranica za korisničko kontrolisanje diode uz pomoć dva dugmeta.

```
<!--index.html-->
<!DOCTYPE html>
<html lang="sr">
<head>
    <meta charset="UTF-8">
    <title>WebSocket LED kontrola</title>
</head>
<body>
    <h2>WebSocket upravljanje LED diodom</h2>
    <button onclick="ledOn()">UKLJUČI LED</button>
    <button onclick="ledOff()">ISKLJUČI LED</button>
    <p id="status">Status: nepoznat</p>
<script>
    const socket = new WebSocket("ws://localhost:8765"); // ili IP Raspberry
Pi-ja
    socket.onopen = () => {
        document.getElementById("status").innerText = "Status: povezano";
    };
    socket.onerror = () => {
        document.getElementById("status").innerText = "Status: greška u
konekciji";
    };
    socket.onmessage = (event) => {
        document.getElementById("status").innerText = "Status: " + event.data;
    };
    function ledOn() { socket.send("ON"); }
    function ledOff() { socket.send("OFF"); }
</script>
</body>
</html>
```

Nakon toga sledi pokretanje python skripte za pokretanje Web Servera:

```
mihajlo@DESKTOP-BAFHUP9:~$ python3 ws_server.py
```

U Thonny okruzenju dobijamo u shellu poruku kao:



U fajlu HTML + JavaScript zameniti (ukoliko želite da ne bude localhost):

ws://IP_ADRESA_RPI:8765

npr.

ws://192.168.1.50:8765

Funkcionalnost projekta:

Što se tiče python skripte, na početku imamo asyncio modul za pisanje asinhronog koda, kao i websockets biblioteku odnosno modul za pokretanje i rad sa web socketom uz pomoć python-a kao i Rpi biblioteku. Nakon konfiguracije samih GPIO pinova i postavljanje Led diode na logički nivo 0, sledi definisanje asinhrone funkcije koja kao parametar prima websocket. Asinhrono programiranje predstavlja način pisanja programa tako da je moguće da više taskova odnosno zadataka u programu bez blokiranja istog, odnosno tako da se više taskova može izvršavati paralelno ili nezavisno jedan od drugog. Ovde je to jako bitno s obzirom da se vrši dvosmerna komunikacija izmedju klijenta i servera pa nam kod http dela projekta nije bio potreban asinhroni kod s obzirom da se komunikacija vršila sinhrono odnosno jednosmerno. Unutar led_control funkcije iteriramo asinhrono kroz svaku poruku unutar websocketa i vršimo ispit ukoliko je poruka ON, onda postavljamo Led diodu na logički nivo 1 i zatim čekamo poruku da se pošalje bez blokiranaj ostatka programa, i dok se poruka šalje, drugi taskovi odnosno zadaci mogu raditi paralelno s tim. Ovde se šalje poruka na web socketu koju obradjujemo unutar JS-a. Ukoliko je poruka OFF onda radimo isto samo što postavljamoe led diodu na logički nivo 0. Unutar asinhrone main funkcije, vršimo serviranje websocketa tako da sluša zahteve klijenta. Kada se klijent poveže poziva se led_control funkcija i server radi na portu 8765. U delu await asyncio.Future() se kreira beskonačna asinhrona operacija koja drži server aktivnim dok se ne prekine totalno veza. Nakon toga sledi obrada izuzetaka gde u try bloku pokrećemo main funkciju, u except bloku ispisujemo da je server zaustavljen ukoliko se baci KeyboardInterrupt izuzetak i u finally bloku vršimo postavljanje GPIO pinova na stanju visoke impedanse.

Što se tiče Html/js koda, sastoji se iz osnovne html strukture kao i svaki html kod. U Body tagu imamo heading, jedan paragraf i dva dugmeta koji imaju event onclick koji poziva funkcije ledOn i ledOff. Nakon toga imamo script tag u kome pišemo naš js kod.

Kreiramo konstantu odnosno objekat socket klase **WebSocket** koji kao argument konstruktoru prosledjuje adresu ws://IP_ADRESA_RPI:8765". Umesto IP_ADRESA_RPI, postavljeno je localhost i to omogućava rad lokalno na samom mikroračunaru. Kao dodatak, imamo za socket dve funkcije kada se otvara socket i kada ima greška, gde ukoliko ima greške i pri ispisu statusa se ispisuje u paragrafu.

Za samu poruku ispisujemo takodje status. Nakon toga definišemo funkcije ledOn koja salje preko socketa poruku ON i ledOff koja preko socketa salje poruku OFF. Ovime ostvarujemo to da se jednom otvara komunikacija dvosmerna izmedju klijenta i servera, i nakon toga dok se server ne zatvori je moguće slanje podataka i dobijanje responsa sa web servera u realnom vremenu za razliku od http protokola.

